IIT MADRAS BS DEGREE PROGRAMME

Introduction to C Programming

Language fundamentals, memory, pointers, strings, allocation, files and debugging

Converted from the corresponding Notion study guide. This editable LaTeX source includes typeset equations, tables, recreated TikZ flowcharts, and PGFPlots graphs. The notes are syllabus-aligned study material, not a verbatim transcript of every lecture.

Open original Notion study notes

Compiled 29 October 2026

Contents

1 Course overview and computer model 2

2 Memory and the first program 2

3 Binary, hexadecimal, types and expressions 2

4 Branches and loops 3

5 Functions, scope and recursion 3

6 Pointers and arrays 3

7 Strings 4

8 Structures, unions, enumerations and typedef 4

9 Dynamic memory 4

10 File I/O and multi-file programs 5

11 Bitwise operations and masks 5

12 Common defects 6

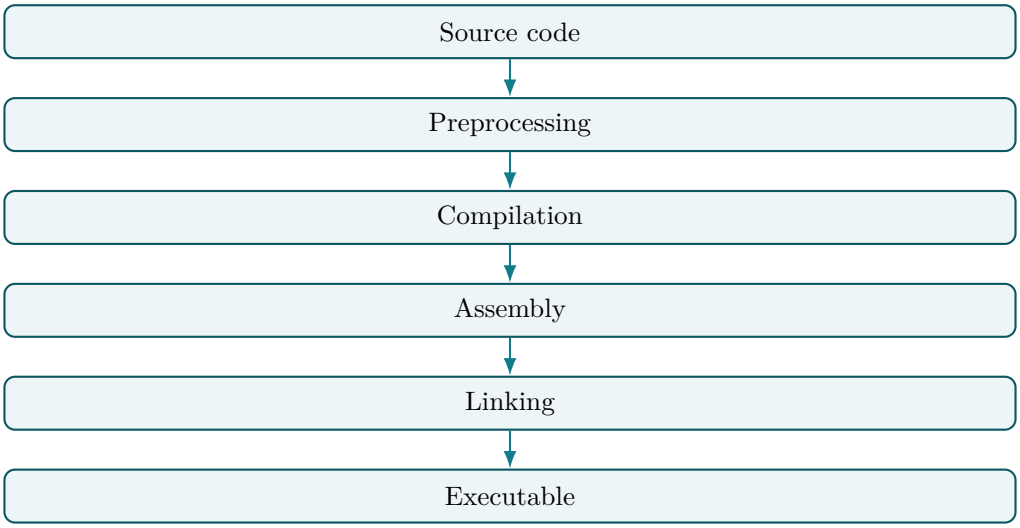
13 Practice and quick reference 6

14 Revision checklist 6

15 Source and scope 6

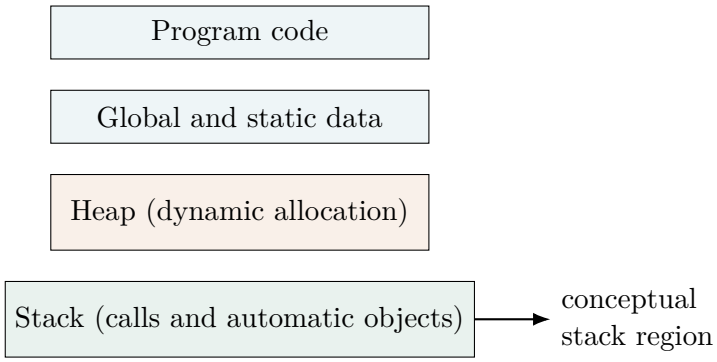
1 Course overview and computer model

C is a compiled, procedural systems language. This guide follows the course progression from program basics and number representation to expressions, control flow, functions, pointers, arrays and strings, structures, dynamic memory, files, bit operations, and multi-file projects.



2 Memory and the first program

A program transforms input/state into output. A simplified process memory view contains code, global/static objects, a stack commonly used for automatic objects and calls, and a heap used for dynamic allocation. Exact layout is operating-system and implementation dependent.



A minimal program:

```c
#include <stdio.h>

int main(void) {
    printf("Hello, world!\n");
    return 0;
}
```

3 Binary, hexadecimal, types and expressions

An unsigned binary value is a weighted sum $\sum_{i=0}^{n-1} b_i 2^i$. One hexadecimal digit represents four bits. For example, $101101_2 = 32 + 8 + 4 + 1 = 45_{10} = 2D_{16}$. Two's complement with n bits represents

signed values from -2^{n-1} to $2^{n-1}-1$. C types control value representation and operations. Integer division truncates toward zero; floating-point division retains fractional values if an operand is floating-point. Integer overflow, conversions, and signedness require careful analysis.

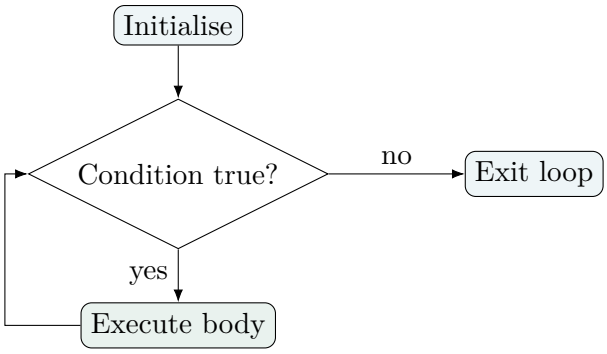
Worked example

`7 / 2` evaluates to integer 3, while `7.0 / 2` evaluates to 3.5.

4 Branches and loops

`if`/`else` selects a branch; `switch` selects among cases; loops repeat a block. A counted loop typically has initialisation, condition, body, and update. Check boundary conditions to avoid off-by-one errors.

```c
int total = 0;
for (int i = 1; i <= 4; ++i) {
    total += i;
}
printf("%d\n", total); /* prints 10 */
```



5 Functions, scope and recursion

A function declaration (prototype) states the interface; its definition supplies the body. C passes arguments by value. To let a function modify a caller's object, pass a pointer to it. Scope determines where an identifier is visible; lifetime determines how long the object exists. Recursion requires a base case and progress toward it.

```c
int square(int x); /* prototype */

int square(int x) {
    return x * x;
}
```

6 Pointers and arrays

A pointer stores an address of an object or function of a compatible type. `&x` obtains an object's address; `*p` accesses the pointed-to object. Pointer arithmetic is constrained by array object boundaries. A pointer must be valid before dereferencing.

```c
int n = 5;
int *p = &n;
```

```
*p += 3;
printf("%d\n", n); /* prints 8 */
```



An array contains contiguous elements indexed from zero. For `int a[3] = {4, 5, 6};`, the valid indices are 0, 1 and 2; `a[3]` is out of bounds. The size of an array is not the same as the size of a pointer.



7 Strings

A C string is a sequence of characters terminated by the null character `'0'`. The literal `"cat"` has four array elements when stored as `char s[]`: three letters and the terminator. `sizeof s` is 4 in that context; `strlen(s)` is 3.

```
char s[] = "cat";
/* c  a  t  \0 */
if (strcmp(s, "cat") == 0) {
    puts("match");
}
```

Use `strcmp` to compare string contents, not `==` (which compares pointer values in common expression contexts). Avoid buffer overflows; validate lengths and prefer bounded input approaches such as `fgets`.

8 Structures, unions, enumerations and typedef

A `struct` groups fields which can all be present at once. A `union` overlays alternative members in shared storage. An `enum` defines named integer constants. `typedef` introduces an alias for a type. Use the arrow operator `->` to access a field through a pointer to a structure.

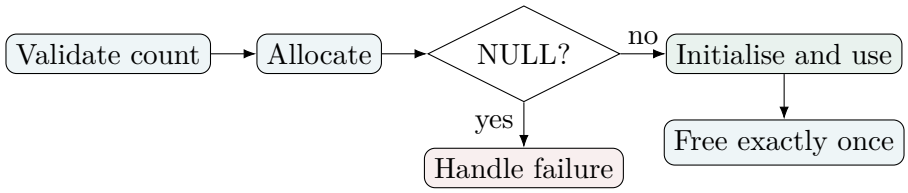
```
typedef struct {
    unsigned id;
    double value;
} Reading;

Reading r = {.id = 1, .value = 2.5};
Reading *ptr = &r;
printf("%.1f\n", ptr->value);
```

9 Dynamic memory

`malloc` allocates uninitialised storage; `calloc` allocates and zero-initialises bytes; `realloc` attempts to resize an allocation; `free` releases it. Check allocation results and arithmetic used to calculate size. After freeing, do not dereference the old pointer; avoid double-free and use-after-free.

```c
size_t count = 10;
int *values = malloc(count * sizeof *values);
if (values == NULL) {
    /* handle allocation failure */
    return 1;
}
/* Initialise values before reading them. */
free(values);
values = NULL;
```



10 File I/O and multi-file programs

Typical file workflow: open, check the returned pointer, read/write, check operation results, and close. Use `fgets` for line-based input. Organise declarations in headers and definitions in source files; header guards prevent repeated inclusion. Link all required translation units.

```c
FILE *fp = fopen("data.txt", "r");
if (fp == NULL) {
    perror("data.txt");
    return 1;
}
char buffer[256];
while (fgets(buffer, sizeof buffer, fp) != NULL) {
    fputs(buffer, stdout);
}
if (ferror(fp)) {
    /* handle read failure */
}
fclose(fp);
```

11 Bitwise operations and masks

Bitwise AND, OR, XOR, NOT, left shift and right shift operate on integer representations. A mask selects bits for setting, clearing or toggling. Use unsigned types for bit-manipulation patterns where appropriate and be careful with shift widths and signed shifts.

```c
unsigned flags = 0u;
unsigned mask = 1u << 3;
flags |= mask;    /* set bit 3 */
flags &= ~mask;   /* clear bit 3 */
flags ^= mask;    /* toggle bit 3 */
```

12 Common defects

Common trap

Uninitialised reads; out-of-bounds access; invalid, null or dangling pointer dereference; use-after-free; double-free; returning a pointer to a local automatic object; incorrect format specifiers; unchecked allocation or file errors; unsafe input; integer overflow; data races in concurrent code; and reliance on undefined behaviour.

Debug by reproducing with a minimal case, reading the first meaningful compiler warning, checking initialisation and bounds, verifying pointer lifetime/ownership, inspecting return codes, and rerunning regression tests after fixes. Enable warnings and use sanitizers/debuggers when available.

13 Practice and quick reference

Task	Safe pattern
Print an integer	`printf("%d\\n", n);`
Check integer input	`scanf("%d", &n)== 1`
Counted loop	`for (size_t i=0; i<n; ++i)`
Address / dereference	`&object` / `*ptr`
String comparison	`strcmp(a,b) == 0`
Allocate	`malloc(count * sizeof *ptr)` after size checks
Read line	`fgets(buffer, sizeof buffer, fp)`
Compile multiple files	`gcc main.c helper.c -o app`

14 Revision checklist

- Explain preprocessing, compilation, assembly, linking and runtime.
- Convert binary, decimal and hexadecimal; reason about signed representations.
- Trace expressions, branches, loops, function calls and recursion.
- Draw pointer and array memory layouts; respect object lifetime and bounds.
- Manage strings with sufficient storage for the null terminator.
- Use structures, dynamic memory and file I/O with failure checks.
- Understand bit masks, multi-file builds, compiler warnings and debugging.

15 Source and scope

Source: Introduction to C Programming – Complete Study Notes and Visual Revision Guide. Official outlines: https://study.iitm.ac.in/ae/course_pages/BSCS1101.html and https://study.iitm.ac.in/es/course_pages/CS1101.html. The code is illustrative and should be compiled under the target compiler/C standard; language details and behaviour may vary by implementation where the standard permits it.